Technical Documentation: Neural Network Simulation Bridging Mechanisms

Documentation Report

April 7, 2026

1 evaluate_select_place()

The `evaluate_select_place()` function acts as the core bridging mechanism between a point-neuron network simulation (like NEST) and a detailed 3D multi-compartment simulation (like LFPy). Its primary job is to take a post-synaptic neuron and a list of its pre-synaptic partners, statistically generate a ”cloud” of expected synapses, and then find a 3D physical morphology that can realistically host those synapses without violating biological constraints.

Here is a detailed, step-by-step breakdown of exactly what it does and how it operates.

Phase 1: Generating the ”Abstract Synapse Cloud”

Before touching any 3D models, the function calculates where synapses ought to be in space based purely on statistics and overlapping probability fields.

- **Iterating Pre-Synaptic Partners:** It loops through every pre-synaptic neuron designated to connect to the current post-synaptic cell (provided via `pre_partners_matrix`).
- **Overlap Calculation:** For each partner, it takes the spatial coordinates and biological types of both cells. Using pre-calculated density maps (loaded from `density_maps_dir`), it computes the intersection of the pre-synaptic cell’s axonal arbor probability and the post-synaptic cell’s dendritic arbor probability.
- **Probabilistic Placement:** Based on this overlap, it rolls the dice to place the required number of synapses in 3D space (`calculate_synaptic_overlap_locations` and `distribute_synapses_proba`
- **Result:** It builds a dictionary (`all_synapse_locations`) mapping each pre-synaptic neuron ID to a list of abstract [x, y, z] coordinates.

Phase 2: Layer Capacity Profiling (The ”Shape” Check)

Once the abstract cloud is generated, the function determines what kind of physical tree is required to support it.

- **Z-Coordinate Sorting:** It flattens all the abstract 3D points and looks exclusively at their vertical Z coordinates.
- **Layer Binning:** Using the defined `layer_bounds` (e.g., L1, L2/3, L4, etc.), it counts exactly how many synapses fall into each cortical layer.
- **The Profile:** This creates a strict physical requirement profile (e.g., ”The chosen morphology must have at least 15 synapse locations in L4 and 5 in L5”).

Phase 3: The Search Loop & Two-Pass Validation

The function now grabs a randomized list of candidate 3D morphologies (`morph_paths`) and evaluates them one by one. To be accepted, a morphology must pass two rigorous tests.

Pass 1: The Capacity Check

- For a given candidate morphology, the function loads its pre-mapped database of physical synapse locations (`mapped_synapses.csv`).
- It shifts and rotates these physical coordinates to match the current post-synaptic soma position.
- It checks the available physical sites against the Layer Profile generated in Phase 2.
- **Outcome:** If the morphology's dendrites don't reach high enough into L4 to host the 15 required synapses, it is instantly rejected, and the loop moves to the next candidate.

Pass 2: The Spread Check (Anti-Clumping)

- If a morphology is big enough and reaches the right layers, the function attempts to physically attach the abstract cloud to the tree.
- **Snapping:** It uses a KDTree (`map_abstract_synapses_to_segments`) to find the absolute closest physical dendritic segment (`lfpy_idx`) for every single abstract coordinate in the cloud.
- **Clumping Validation:** It loops through the newly snapped segments, grouped by pre-synaptic partner. If a pre-synaptic neuron provides multiple synapses (e.g., 5 synapses), it checks if all 5 ended up snapped to the exact same dendritic segment `segment`.
- **Outcome:** Multiple synapses from the same partner clumping onto a single tiny segment is biologically unrealistic and causes severe simulation artifacts. If the unique number of targeted segments is exactly 1 (checked via `len(set(lfpy_indices)) == 1`), the morphology is rejected for "clumping," and the search loop continues.

Phase 4: Finalization and Output

- The moment a morphology successfully passes both the Capacity Check and the Spread Check, the loop stops.
- If the `plot_result` flag is triggered, it extracts the 3D skeleton directly from the `.hoc` file using NEURON (`h`) to prepare for visualization.
- It returns the winning morphology ID, the dictionary of successfully snapped 1D segment indices ready for LFPy injection, and the abstract 3D cloud for downstream debugging and visual plotting.
- If the loop exhausts all candidates without finding a suitable fit, it safely breaks and returns empty structures, preventing infinite loops.

2 cell_MorphSelect (The Biological Gatekeeper)

The Core Concept: This function receives the ”demand profile” generated above and acts as the supply checker. It combs through a folder of candidate cell morphologies and physically measures them to see if they can survive the biological requirements of the simulation.

Step-by-Step Logic:

- **Randomize the Candidates:** It shuffles the list of candidate morphology paths. This ensures that if you are building a network of 1,000 Layer 5 cells, they don’t all get assigned the exact same morphological clone (assuming multiple candidates fit the criteria).
- **Load and Clean the Database:** It opens the `mapped_synapses.csv` for a candidate. This CSV contains every valid point on that specific cell’s dendritic tree where a synapse could theoretically attach. It includes bulletproof data-cleaning steps (stripping whitespace, renaming columns) to prevent pandas from crashing on poorly formatted files.
- **The Critical Spatial Shift:** The coordinates in the `.csv` are usually in local space (meaning the soma is at 0,0,0). The function applies `shifted_z = syn_df['z'] + target_z_pos`. This projects the cell into the global cortical volume so its branches can be compared accurately against the global layer boundaries.
- **The Layer Capacity Test:** The function loops through the required profile. If the profile says ”I need 42 synapses in Layer 4,” the script masks the shifted Z-coordinates to count exactly how many physical dendritic segments this specific cell has inside the Layer 4 boundaries.
- If `available_synapses >= required_synapses`, it moves to check the next layer.
- If `available_synapses < required_synapses`, the cell fails immediately. The function rejects it and loads the next candidate.
- **Return the Winner:** The moment it finds a morphology that passes the test for every single layer, it halts the search and returns that morphology’s ID.

Why it’s brilliantly designed: It prevents ”biological clipping.” If a simulation assigns a connection from a high Layer 2/3 cell, but you randomly selected a post-synaptic morphology whose apical tuft was stunted and didn’t reach Layer 2/3, the snapping function would later fail (or snap the synapse wildly out of place). `cell_MorphSelect` guarantees that the physical tree matches the mathematical blueprint.

3 map_abstract_synapses_to_segments (The Batch Finalizer & Translator)

The Core Concept: Neural simulation environments like NEURON or LFPy do not natively simulate cells in 3D Cartesian space (x, y, z). Instead, they simulate cells as a 1D topological tree made of ”segments” or ”compartments” (e.g., ”inject current into branch #45, segment #2”).

By the time the script reaches this function, you have a massive list of floating 3D coordinates. This function’s sole purpose is to map those physical 3D coordinates to their exact 1D topological segment IDs (`lfpy_idx`) so the electrical simulation can actually run.

Step-by-Step Logic:

- **Iterate by Pre-Synaptic Source:** The function receives `abstract_synapses_dict`, which groups all the generated 3D floating coordinates by the ID of the pre-synaptic cell that created them. It loops through this dictionary one pre-synaptic ID (`pre_idx`) at a time.
- **Call the Snapping Function (Batch Processing):** For a specific pre-synaptic cell, it takes its bundle of 3D coordinates and feeds them into `snap_to_closest_actual_synapses`. This nested function does the heavy geometric lifting: it translates the floating global coordinates back to the local space of the chosen post-synaptic cell, builds a spatial KDTree, and snaps the floating points to the nearest physical dendritic branch.
- **Extract the Simulation Index (lfpy_idx):** This is the most critical step of the function. The snapping function returns a Pandas DataFrame (`snapped_df`) containing all the biological info about the matched synapses. `map_abstract_synapses_to_segments` deliberately strips away the (x, y, z) coordinates, the errors, and the metadata. It isolates exactly one column: `lfpy_idx`.

  Code snippet: `lfpy_indices = snapped_df['lfpy_idx'].astype(int).tolist()`

  It forces them into integers just in case pandas interpreted them as floats (e.g., 45.0 to 45), ensuring strict compatibility with NEURON.
- **Error Handling & Fallbacks:** If the snapping function fails for a specific connection (e.g., the coordinates were too far away, or the DataFrame returned empty), the script doesn't crash. It catches the error, prints a warning ("Snapping failed or returned empty..."), and assigns an empty list `[]` to that pre-synaptic cell, allowing the rest of the network to continue building.
- **Return the Final Dictionary:** It returns a highly condensed dictionary. Instead of complex 3D coordinate matrices, it simply outputs a mapping like `{pre_cell_101: [45, 46, 102], pre_cell_102: [8, 12]}`.